

Livepeer in 2026: The Decentralized Video and AI Stack

Livepeer has evolved from a pure video-transcoding protocol into a five-layer decentralized compute stack spanning video, real-time AI inference, and creative agent tooling. As of April 2026 the `livepeer` GitHub organization hosts **173 public repositories**, anchored by `go-livepeer` (583 stars, the reference Go node), `lpm` (the media server), `protocol` (the Arbitrum-deployed Solidity contracts), `ai-runner` / `ai-worker` (the AI inference runtime), `studio` (the hosted video product), and `ui-kit` (React video primitives). New additions in the last six months — `storyboard`, `naap`, `livepeer-data-mcp`, `pytrickle`, `ComfyUI-Stream-Pack`, `coordination`, and a parallel docs v2 hierarchy under `/v2/` — signal a decisive pivot toward **real-time AI video, agent workflows, and plugin-based platform architecture**. The old "web3 live streaming" framing is gone; the network now markets itself as an **open, GPU-dense, decentralized compute layer that happens to be best-in-class at video**.

This report maps where Livepeer stands today: its product layers, the canonical developer choices, what the AI runtime actually supports, and the community tooling and governance scaffolding behind it.

The five-layer developer stack

Livepeer's own v2 documentation (in `livepeer/docs` under `/v2/developers/concepts/developer-stack.mdx`) now explicitly describes the ecosystem as **five distinct layers**, and this framing is the single most useful mental model for anyone evaluating the project. Each layer trades control for convenience.

At the top, **Studio** (`livepeer/studio`, 85 stars, TypeScript) is the hosted video product operated by Livepeer Inc — livestreaming, VOD, transcoding, the embeddable player, webhooks, and analytics behind an API key. `livepeer` It targets developers who want managed video and don't care about the decentralized substrate. **Daydream** (built on the open-source `comfystream` ComfyUI plugin by yondonfu, 124 stars) is Studio's AI equivalent: a hosted real-time AI video API at `daydream.live` that applies text-prompt effects to live webcam feeds, `livepeer+2` plus an SDK (`sdk.daydream.monster`) used by downstream apps like Storyboard.

The **AI Gateway API** is the middle layer — a direct REST interface to the Livepeer AI inference network, callable without Studio or Daydream. Multiple gateway providers exist, including Livepeer Studio Gateway (production) `livepeer` and the free Livepeer Cloud Community Gateway. `livepeer` Below that, **ComfyStream** is the open-source ComfyUI plugin that turns ComfyUI workflows into real-time video pipelines (`The Cloud SPE`) — Daydream itself runs on ComfyStream, `livepeer` `Daydream` and Livepeer extends it with the new `ComfyUI-Stream-Pack` (20 stars, Feb 2025) that adds real-time I/O nodes for video, audio, and tensor streaming. `livepeer`

At the base sits the **protocol layer**: `go-livepeer` (Broadcaster/Orchestrator/Transcoder node software), `livepeer/protocol` (Delta-era Solidity contracts on Arbitrum, 153 stars, 46 forks), `ai-worker` and `ai-runner` (GPU inference execution), and `lpm` (the media server, 283 stars). Running at this layer means **operating nodes, earning LPT, or contributing**

new pipelines. The docs explicitly describe a "graduation path" mirroring Heroku → AWS → own infrastructure: start on Studio/Daydream, move to direct AI Gateway calls, eventually self-host a gateway to cut per-inference cost. (Livepeer)

The AI runtime is now the center of gravity

Five years ago Livepeer was transcoding-only; today the (ai-runner) repo (24 stars, 96 open issues, last updated March 2026) defines the product. It is a **containerized FastAPI service** that loads models into GPU memory and exposes REST endpoints to the (go-livepeer) AI worker. (GitHub) Inspection of (runner/src/runner/pipelines/) reveals the current production pipeline set: **audio-to-text, image-to-image, image-to-text, image-to-video, LLM, segment-anything-2, text-to-image, text-to-speech, upscale, frame-interpolation, and — critically —** (live_video_to_video.py) (12.4 KB, the largest pipeline file), which is the LV2V (live video-to-video) path that powers Daydream and Storyboard's real-time AI effects. The runner uses (uv) for dependency management, Hugging Face for checkpoint downloads ((dl_checkpoints.sh)), and has a Modal deployment path ((modal_app.py)) for serverless GPU bursting.

Around the runner, two newer Livepeer-owned Python packages fill in the real-time plumbing. (pytrickle) (3 stars, July 2025) implements the trickle protocol for low-latency streaming ingress/egress (GitHub) with a (FrameProcessor) base class, (GitHub) PyTorch tensor operations, async video+audio, and a built-in HTTP control API for dynamic parameter updates at up to 60 FPS. (ComfyUI-Stream-Pack) exposes ComfyUI nodes that read from and write to these trickle streams, letting creators build custom real-time workflows visually. Together they represent Livepeer's answer to "how do you let an arbitrary ComfyUI graph process a live webcam feed at sub-second latency on a decentralized network" — and that is the question Daydream commercializes.

The AI surface area is sprawling. The (livepeer-ai-python) SDK and the v1 API reference list formal endpoints for 9 pipelines, but (storyboard)'s README claims the Daydream SDK exposes **40+ models** via a BYOC (Bring Your Own Container) orchestrator — flux-dev, flux-schnell, recraft-v4, seedream, nano-banana, veo, seedance, ltx, pixverse, kling for I2V, chatterbox-tts, inworld-tts, grok-tts, trip03D, and real-time LV2V via the Scope pipeline. The gap between "official documented pipelines" and "what's actually callable through a gateway today" is one of the ecosystem's biggest documentation problems.

SDKs, APIs, and the hosted product surface

Livepeer ships first-party SDKs for TypeScript/JavaScript ((livepeer-js)), Python ((livepeer-python)), Go ((livepeer-go)), Java ((livepeer-java)), Ruby ((livepeer-ruby)), and PHP ((livepeer-php)), plus a dedicated (livepeer-ai-python) package for AI endpoints. All are Speakeasy-generated from an OpenAPI spec, giving consistent ergonomics: bearer-token auth, (GitHub) retries with backoff, pluggable HTTP clients, (GitHub) and standalone tree-shakable functions. (npm) The surface covers (accessControl), (asset), (generate) (AI), (metrics), (multistream), (playback), (session), (stream), (task), (transcode), and (webhook) — with the (room) namespace fully deprecated (live video rooms were cut). The React-side story is

`livepeer/ui-kit` (71 stars), `npm` which provides composable Player and Broadcast primitives with WebRTC/HLS, low-latency, and WAI-ARIA built in.

On top of the SDKs, **Livepeer Studio** (`livepeer/studio`) is the monolith: auth, billing, API keys, multistream, recording, thumbnails, `GitHub` signing keys for JWT access control, `GitHub` Prometheus metrics export, and the webhook/VOD task runner (`livepeer/task-runner`). The **Catalyst** media server stack (`livepeer/catalyst`, 24 stars; plus `catalyst-api`, `catalyst-uploader`, `catalyst-node`) is the decentralized successor to MistServer integration, handling ingest, segment upload to S3/Storj/IPFS, and the network-side media plane. The `studio-cli`, `livepeer-monitoring`, `current-viewers`, and `player-metrics` repos round out the operator experience.

Community infrastructure and the orchestrator economy

The orchestrator side of the network has a **rich third-party tooling ecosystem that Livepeer Inc does not build directly**. Rick Staa (`rickstaa`) maintains `livepeer-exporter` (Prometheus metrics), `livepeer-reward-watcher`, and `livepeer-income-reports` (tax/earnings scripts), plus the Livepeer AI Dune dashboard. **Stronk-Tech** (Captain Stronk) runs a large sub-ecosystem: `OrchestratorTracker`, `LivepeerPerformanceAI`, `LivepeerPerformanceTranscoder`, `LivepeerExplorer`, `LivepeerStats`, `MistLoadLivepeer`, `StronkAI` (jobs dashboard), plus the `StronkController`/`StronkLive`/`GitHub` `StronkPlayer` stack which appears to be a **stake-for-access livestreaming platform built on Livepeer as a dependency, not a Livepeer product**. Titan Node, `GitHub` Video Miner, Livepool, Grant Node Pool, and Open-Pool run orchestrator pools, and Tenderize `GitHub` provides liquid staking for LPT. `GitHub` +2

Livepeer's own network-side governance tooling lives in `LIPs` (Livepeer Improvement Proposals, 8 stars), the `grants` repo (now deprecated in favor of a Notion page), `GitHub` `community-governance`, and the brand-new `coordination` repo (Oct 2025) — which replaces meetings with a **GitHub-issue-driven async-first process** where each team submits a sub-issue 24 hours before a scheduled meeting, letting the owner prepare an agenda focused on decisions rather than status. The `roadmap` repo (Sept 2025) uses GitHub Projects as a community-visible roadmap. `arbitrum-lpt-bridge` (18 stars) handles LPT movement between Ethereum L1 and Arbitrum `Livepeer` One, `GitHub` where the protocol has lived since the 2022 migration.

The 2026 pivot: agents, platforms, and data

Three repos created in the last eight months signal where Livepeer Inc is actually investing engineering effort, and it is **not incremental improvements to hosted video**.

`naap` (**Node-as-a-Platform**), Feb 2026. A plugin-based platform for the Livepeer AI Compute Network built on Next.js 15, React 19, and a UMD-bundle micro-frontend architecture. A Next.js shell dynamically loads independently-built plugins from a CDN, each with its own Express backend and isolated PostgreSQL schema. Twelve plugins ship today. Independent teams can develop, deploy, and operate plugins without central coordination — a deliberate bet on **lazy-consensus governance and team autonomy** rather than a single monolithic product.

[storyboard](#)), **April 2026**. An AI-agent creative workspace (live at [storyboard-rust.vercel.app](#)) that orchestrates 40+ models into text-to-image, I2V, talking-head, lipsync, 3D, and real-time LV2V workflows on an infinite canvas. Its architecture is a clean three-layer SDK: [@livepeer/agent](#) (provider-agnostic agent runtime with swappable Gemini/Claude/OpenAI/Livepeer backends), [@livepeer/creative-kit](#) (reusable canvas + command router + capability resolver with automatic model fallback), and the application itself. **The Livepeer provider routes all LLM calls through [/api/llm/chat](#) so one Daydream API key drives every model** — a hosted-aggregator play that makes Livepeer the default inference backend for a full agent stack.

[livepeer-data-mcp](#)), **Feb 2026**. An MCP server giving Claude/Cursor plain-English access to Livepeer's internal ClickHouse data warehouse (via dbt semantic models) and PostHog analytics. It is deployed via ArgoCD at [data-mcp.sre.livepeer.technology](#) and is an **internal-operations tool**, not a customer product — but its existence indicates Livepeer now runs a full modern data stack (ClickHouse, dbt, PostHog, Vault-managed secrets, Cloudflare tunnels, Kubernetes on Vercel+managed infra).

Alongside these, [embody](#), [frameworks](#), and [streamplace](#) appear as **first-class solution verticals** in the v2 docs ([/v2/solutions/](#)), each with overview, community, and changelog pages. [streamplace](#) (the external [streamplace/streamplace](#) repo, 213 stars, "Live video for the AT Protocol") [Livepeer Forum](#) is the Bluesky-native live video project, positioning Livepeer as the **infrastructure layer for the decentralized social video stack**.

Documentation, discoverability, and the hard edges

Livepeer maintains the docs at [docs.livepeer.org](#) through the [livepeer/docs](#) repo (44 stars, 74 forks — unusually high fork count, reflecting active external contribution). The most striking finding in the docs is the **parallel [/v1/](#) and [/v2/](#) hierarchies**. The v1 tree is the battle-tested operator/developer reference: API endpoints, AI pipelines, orchestrator setup, delegator guides, SDK references, and the [awesome-livepeer](#) mirror. The v2 tree is a ground-up reorganization around **personas and solutions** — [about/](#), [community/](#), [developers/](#), [gateways/](#), [orchestrators/](#), [delegators/](#), [solutions/](#) (daydream, embody, frameworks, livepeer-studio, streamplace), plus an [internal/rfp/](#) section with problem statements, deliverables, and outcomes that suggests the docs team is running a formal RFP process against itself. The v2 developer tree surfaces new concepts ([byoc.mdx](#), [workload-fit.mdx](#), [sdk-gateway.mdx](#), [pytrickle.mdx](#), [ai-on-livepeer.mdx](#)) that simply do not exist in v1.

The migration is incomplete. Several things are genuinely hard to find from a cold start: which gateway endpoints are production-grade versus experimental, what the current **real AI pipeline support matrix** looks like versus the 40-model Daydream claim, how LV2V pricing works, and whether [streamplace](#) is Livepeer-operated or community-operated (it lives under [streamplace/streamplace](#), not [livepeer/streamplace](#)). Expect the v2 docs to consolidate over the next quarter — the [internal/rfp/](#) pages and the [coordination](#) repo both indicate organized effort to close these gaps.

For anyone choosing a layer to build on today, the decision tree is clearer than it has ever been. Build on **Studio** if you want managed video with API keys and a dashboard and are happy trading control for speed. Build on **Daydream** if you need real-time AI video effects and don't want to run ComfyUI yourself. Call the **AI Gateway API directly** (via the community gateway for free, the Studio gateway for production) if you need control over which orchestrators run your jobs and want to avoid Daydream-specific branding. Run **ComfyStream + pytrickle** locally or on RunPod/TensorDock if you need custom workflows, fine-tuned LoRAs, or BYOC pipelines. Run a **gateway node** if your volume justifies per-inference cost control. Contribute to `go-livepeer`, `ai-runner`, or `protocol` if your goal is to expand what the network itself can do.

The most important shift to internalize: **Livepeer is no longer "web3 live streaming."** It is a **decentralized GPU compute network** whose flagship workloads are real-time video-to-video AI, agent-driven creative tools, and the plumbing under federated social video. The protocol, LPT token, orchestrator economy, and Arbitrum contracts are still there (`GitHub +3`) — but they are now the substrate under a much larger surface that looks more like a cloud AI provider than a blockchain.

Key takeaways

The next 12 months will be defined by whether the v2 docs and the NaaP plugin platform successfully abstract away the complexity the ecosystem has accumulated. **The engineering exists; the narrative is catching up.** Storyboard, NaaP, and the data-MCP together demonstrate that Livepeer Inc is comfortable operating a modern SaaS stack on top of the decentralized network, and the async-first `coordination` repo suggests the org itself has become plugin-shaped to match. For developers, the practical advice is simple: start at the highest layer that meets your needs, read the v2 `developer-stack.mdx` page before the v1 references, and treat the `awesome-livepeer` list plus the Stronk-Tech dashboards as the real community compass — they reveal what the network actually does in production, which the official docs still only partially describe.